ECE 471: Anomaly Detection with EfficientAD

Jasper Halvorson and Liam Tanner

***Abstract*—EfficientAD is an anomaly detection method proposed in 2024 by K. Batzner et al. which improves upon current algorithms in both accuracy and efficiency [1]. Following the EfficientAD framework, we implemented and trained an anomaly detection model, available in our Google Colab Notebook, using PyTorch to classify abnormalities in images from three separate classes: pasta, screws, and capsules. Our implementation produced anomaly maps and anomaly scores for the test images, and we evaluated the model's performance on the three datasets with AU-ROC and F1 scores. Our model performed well on the screws dataset, moderately well on the capsules, and poorly on the pasta dataset, suggesting that the model's effectiveness depended heavily on the amount and consistency of the data. Overall, this project demonstrated that EfficientAD can be applied to a variety of anomaly detection problems but is challenged by small or inconsistent datasets.**

April 3, 2026

I. Introduction

In this project, we implemented EfficientAD to detect anomalies in three datasets containing images of pasta, screws, and capsules, respectively. Using the training and inference algorithms defined in the EfficientAD paper [1], we implemented our model using PyTorch and trained the networks on A100 GPUs in a Google Colab Environment. Figures 1, 2, and 3 show examples of normal and anomalous images in each dataset.

(a) Normal pasta image (b) Anomalous pasta image

Fig. 1: Normal vs Anomalous Pasta Example

(a) Normal screws image (b) Anomalous screws image

Fig. 2: Normal vs Anomalous Screws Example

(a) Normal capsule image (b) Anomalous capsule image

Fig. 3: Normal vs Anomalous Capsules Example

EfficientAD is an anomaly detection architecture that builds upon the student-teacher approach by adding an autoencoder to the pipeline. In particular, the student tries to mimic both the teacher output (general feature descriptor network) and the autoencoder network, and large differences in the outputs indicate anomalous regions of the input image. EfficientAD can be executed in under a millisecond, while maintaining an average AU-ROC score of 96% across MVTec AD, VisA, and MVTec LOCO (three major anomaly detection datasets) [1]. The algorithm's efficiency and AU-ROC are better than other popular anomaly detection methods such as PatchCore and AST by a considerable margin as shown in Figure 4.

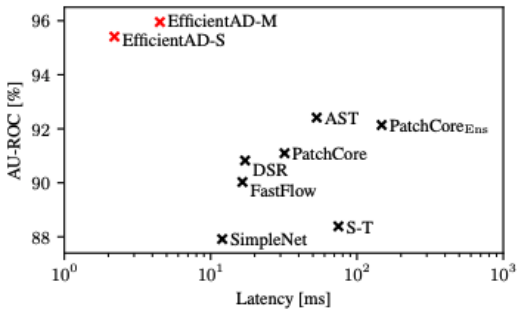


Fig. 4: EfficientAD Comparison to Common Anomaly Detection Techniques [1]

The core innovation of EfficientAD and the main contributing factor to its efficiency is the Patch Description Network (PDN), a lightweight CNN architecture with 4 convolutional layers, 2 average pooling layers and ReLU activations shown in Table I. Both the teacher and student networks are instances of the PDN architecture. This differs from other anomaly detection approaches which often rely on large pre-trained backbone networks such as ResNet. Instead of directly a larger frozen network, the teacher PDN is trained on one (Wide ResNet in our implementation) as it learns to give similar outputs with a more constrained structure. This change drastically improves inference time, while maintaining

enough information to make accurate feature descriptions.

Layer Name	Stride	Kernel Size	Num Kernels	Padding	Activation
Conv-1	1×1	4×4	128	3	ReLU
AvgPool-1	2×2	2×2	128	1	-
Conv-2	1×1	4×4	256	3	ReLU
AvgPool-2	2×2	2×2	256	1	-
Conv-3	1×1	3×3	256	1	ReLU
Conv-4	1×1	4×4	384	0	-

TABLE I: The PDN architecture described in the EfficientAD paper [1]

To detect local anomalies such as changes in texture, our model computes the difference between the student and teacher network outputs on an image. The convolutional autoencoder, as described in Table 8 of the EfficientAD paper [1], is used to detect global anomalies such as missing structures by computing the difference between its output and that of the student on a given image.

As mentioned, our implementation is written, trained and executed in a Google Colab environment. Since EfficientAD requires a large number of iterations to train each network, all of our training was done on A100 GPU runtimes. This significantly improves our training speeds in comparison to using the standard T4 GPUs offered by Google Colab. Once trained, we use the EfficientAD inference algorithm to perform the final anomaly detection on the three separate datasets. The results from each dataset will be individually analyzed and compared using AU-ROC scores and accuracy in Results and Evaluation (Section IV).

II. Literature Review

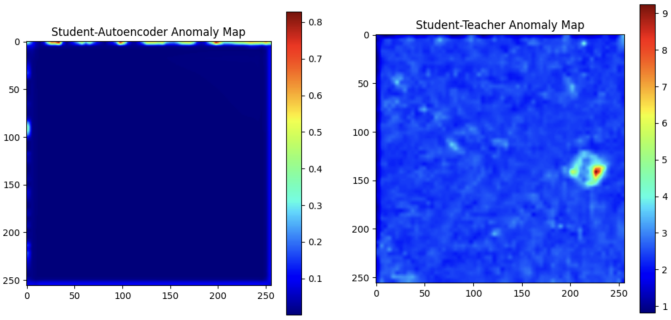
In our initial research, we identified four candidate strategies of anomaly detection in PatchCore [2], Student-Teacher [3], Isolation Forest [4], and PaDiM [5]. Feedback from this initial literature review ruled out Isolation Forest as it is not an effective anomaly detection method for high dimensional data such as images. We noted that PatchCore was a strong anomaly detection method, but it depended on large pre-trained networks and a memory bank, making it computationally expensive to implement. Similar to PatchCore, PaDiM's computation expense and reliance on large pretrained CNN embeddings made it undesirable for this project.

EfficientAD provided a lightweight and robust anomaly detection method, directly applicable to our problem. We decided that building upon the Student-Teacher method with EfficientAD would provide the greatest learning experience and allow us to understand a modern anomaly detector methodology with real-world applications.

III. Implemented Approach

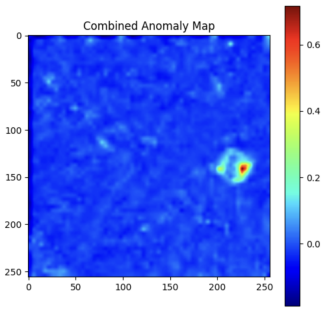
EfficientAD uses a teacher, student, and autoencoder as the three core neural networks in its anomaly detection process. The teacher is a general feature descriptor network, trained to output feature description vectors of size 384 for each 64x64 patch of the 256x256 input image. The autoencoder network compresses input data and reconstructs it while maintaining its key global structures. We train the autoencoder to match the teacher's outputs on normal (non-anomalous) images so that it will greatly differ from the teacher output at test time only on anomalous images. Finally, the student network is trained on normal images to output vectors of size 768 where the first half (size 384) of each mimics the teacher's output for a patch and the second half (size 384) mimics that of the autoencoder.

In inference, as seen in Listing 3, the student and normalized teacher output differences are averaged and interpolated into a 256x256 local anomaly map used to identify anomalous regions (corresponding to high differences). The same strategy is applied to the student and autoencoder outputs to obtain a global anomaly map. We normalize the maps with upper quantile normal difference values from the training and combine them with a weighted average to get an overall anomaly map. The values in this map are used to make a prediction on whether or not an input image is anomalous or not. In particular, the single maximum value in this final anomaly map is used as the overall anomaly score for the image. Below, we show the intermediate and final anomaly maps in an anomalous image. We see that the die is correctly highlighted as an anomaly mostly in the local anomaly map.



(a) Global anomaly map (unnormalized)

(b) Local anomaly map (unnormalized)



(c) Combined Anomaly Map

(d) Original Image

Fig. 5: Anomaly maps and original anomalous pasta image

One key distinction between EfficientAD and other Student-Teacher network frameworks is that the model can detect global anomalies from the student-autoencoder differences in addition to local anomalies from the student-teacher differ-

ences. Using these global features allows for more robust detection of anomalous areas of the images. In Figure 5, we saw a local map dominated example; there are other anomalies which take up larger regions in the image that the global map detects far better than the local map.

A. Teacher Network

The Teacher network is a general purpose patch description network (PDN) built from four convolutional layers as described in the EfficientAD paper [1]; our class implementation of this is available in Listing 4. We train this network to output feature description vectors of size 384 given an image based on the outputs of a highly tuned pretrained Wide ResNet network. In particular, the teacher learns to mimic the behaviour of the ResNet network on images from the ImageNet dataset.

To train the network, we begin by extracting the WideResNet backbone to use as an oracle for the teacher during training. Since we want this oracle to act as a feature descriptor rather than an object classifier, we use layers two and three from the backbone. This results in a 1536 feature output. Since this output must match that of a PDN (the teacher network) we take only the first 384 features. The training dataset for the teacher is a random 500 image subset of Imagenette which itself is a subset of ImageNet. Before training on this data we compute the mean and standard deviation for each of the 384 output channels after running the training set of images through the oracle.

The training loop itself follows Algorithm 3 in the EfficientAD paper [1]. We start by initializing an Adam Optimizer with a learning rate of $1e^{-4}$ and weight decay of $1e^{-5}$. The loop takes 60,000 iterations and processes images in a batch size of 16. The first step is gray scaling each image in the batch with a probability of 10%. Each image is then resized, 512x512 for the oracle input and 256x256 for the teacher. The oracle is then fed the batch of images and outputs the ideal feature map. This map is normalized using the previously described mean and standard deviation. Finally, the teacher attempts to mimic the oracle by creating it's own feature map of the batch. The loss is then computed as the squared difference of the two maps. Finally, the optimizer uses this loss to step.

B. Student and Autoencoder Network Training

Like the teacher, the student network uses the PDN network structure. The autoencoder network, however, is a larger convolutional network, employing 6 layers of encoding (reducing image data) followed by 8 decoding layers. Our autoencoder class is available in Listing 5 and the design comes from Table 8 in the EfficientAD paper [1]. As for training, the autoencoder and student networks are both trained only on normal images. The autoencoder's loss function is based on its squared difference from the teacher output; the student's is from its squared difference with both the teacher and autoencoder networks.

The training strategy is quite involved and comes directly from Algorithm 1 in the EfficientAD paper with minor modifications to match our problem space [1]. Algorithm 1 covers both the autoencoder and student training; we provide our implementation in Listing 2 of the Appendix. We begin by creating an Adam optimizer containing the parameters of the newly created student and autoencoder networks. In the main training loop, we begin by selecting a training (normal) image from the dataset which has been resized and permuted to match our networks, and then compute its resulting student and teacher network outputs. We normalize the teacher outputs using the mean and standard deviation parameters from its training, then compute the student-teacher regularized loss based on the squared difference of the student and teacher outputs. This loss is defined as

$$L_{ST} = mean(DiffST_{0.999}) + (384 \cdot 64 \cdot 64)^{-1} \sum_{i,j \in P}^{384} \left\| S(P)_{ij}^{[l]} \right\|_2^2,$$

where

$$DiffST_{0.999} = Quantile_{0.999}\Big((T(I) - S(I))^2\Big),$$

P is a randomly sampled pretraining image, S is the student network, T is the teacher network (with normalized output), I is an input image, and $Quantile_{0.999}$ filters for the entries with values in the 99.9th percentile. Note that this loss is penalizing student output pixels on normal images in the upper quantile of values only. Since an anomaly score is computed as the maximum value of the anomaly map, this loss strategy will reduce the anomaly score outputted for normal images.

Next, the training image is augmented in one of three randomly selected ways: adjusting the brightness, contrast, or saturation. The autoencoder, student, and teacher are all then run on the augmented image, and the teacher is normalized in the same way as before. We then compute the student-teacher and student-autoencoder squared differences, and take the mean of the 384 element difference vectors as the losses. The total loss is a sum of these and the student-teacher loss on the non-augmented image as shown below.

$$L_{TA} = mean(T(I_{augmented}) - A(I_{augmented}))^2$$

$$L_{SA} = mean(A(I_{augmented}) - S(I_{augmented}))^2$$

$$L_{Total} = L_{ST} + L_{TA} + L_{SA}$$

The final step in the main loop is to take gradients of the loss with respect to the student and autoencoder parameters using PyTorch's *backward()* function, and update the parameters in the optimizer. The second loop in Listing 2 computes squared differences on validation (normal) images with the tuned student and autoencoder networks and creates anomaly maps (normalized difference maps) for student-teacher and student-autoencoder. From these maps, we retrieve 0.9 and 0.995

quantile values which serve as high normal and upper bound normal image values respectively. The training function returns the teacher network, newly trained student and autoencoder networks, teacher mean and standard deviation values, and the 0.9 and 0.995 quantile values for the student-teacher and student-autoencoder anomaly maps. In inference, we use the quantile values to normalize the maps.

C. Model Inference

Our inference implementation in Listing 3 follows Algorithm 2 from the EfficientAD paper [1]. This algorithm creates the anomaly maps as done in the second loop of the training algorithm, normalizes the maps with the quantile values, then returns a single combined student-teacher and student-autoencoder map along with an anomaly score for the image based on the highest value in the combined map. The anomaly score is used to classify the image as anomalous or normal, and the map is used as a visual reference. Examples of anomaly maps were presented in Figure 5.

IV. Results and Evaluation

In Table II, we present our AU-ROC and F1 scores on the pasta, screws, and capsule datasets. These results were obtained following the paper's iteration count and learning rate specifications for the network training in Algorithms 1 and 3 [1]. We observe very good performance with the screw dataset, moderately good performance with the capsules, and poor general performance (AU-ROC) on the pasta with a good F1 score only with a contrived threshold choice.

Dataset	AU-ROC	F1 Score
Screws	0.8800	0.8421
Capsule	0.6823	0.6667
Pasta	0.5000	0.7692

TABLE II: Performance of the anomaly detection model across datasets.

A. Small Datasets

The first and most pertinent consideration for our results is having only a small number of images for training and validation. Using an 80-20 train-validation split, we get the sizes described in Table III.

Dataset	Train	Validation	Test
Pasta	13	4	16
Screws	14	4	15
Capsule	16	4	36

TABLE III: Number of samples in the training, validation, and test sets for each dataset.

We determined this to be the most probable cause for originally observing poor performance on the pasta dataset (AU-ROC score of 0.5) with the paper recommended 70 000 iterations. To mitigate the overfitting effects for the pasta, we reduced the number of iterations to 10 000 and increased the learning rate from 1e-4 to 5e-3 which resulted in a strong AU-ROC score of 0.88. To preserve the integrity of the experiment, we present our AU-ROC and F1 scores on the paper specified parameters since we used these for the other two datasets.

Another possible improvement would be to create new synthetic training and validation data points by applying transformations to the original data. This could help the model generalize better during training.

B. Data Inconsistencies

We noted that some of the non-anomalous test images presented features which were not present in any of the training data. This was challenging for our model to correctly score; the example below highlights a large empty region in the test image, a characteristic not as apparent in any of the training images.

Fig. 6: Test Image Classified as Normal

V. Conclusion

In this project, we implemented EfficientAD with PyTorch to detect anomalies in each of a pasta, screw, and capsule image dataset. We achieved average AU-ROC and F1 scores of 0.78 and 0.7544, respectively, across the screw and capsule datasets indicating a relatively strong classification model on these datasets. One key challenge was the small number of images available for training and validation as this caused noticeable overfitting in the pasta dataset. Modifying the number of iterations and learning rate improved the results, but adding to the training set with augmented versions of the original images would further improve our results.

Appendix A
Implementations

Listing 1: Teacher Training

```python
def train_teacher(teacher, oracle_extractor, distill_images, oracle_mean, oracle_variance, iterations =60000, batch_size=16):
  teacher.train()
  oracle_extractor.eval()
  # Algorithm 3, Line 13: Adam with lr=1e-4 and weight_decay=1e-5
  optimizer = optim.Adam(teacher.parameters(), lr=1e-4, weight_decay=1e-5)
  # Normalize oracle output (Algorithm 3, Line 22)
  mu = oracle_mean.view(1, -1, 1, 1).to(device)
  sigma = oracle_variance.view(1, -1, 1, 1).to(device)
  # Algorithm 3, Lines 14-30: Main training loop
  for iteration in range(1, iterations + 1):
    # Algorithm 3, Lines 16-27: Inner batch loop of 16 images
    # We dont actually need to loop like is done in the algorithm, can process the whole batch at once
    indices = torch.randint(0, len(distill_images), (batch_size,), device=distill_images.device)
    images = distill_images[indices]

    # Algorithm 3, Line 18: Convert to grayscale with probability 0.1
    # Apply this to batch all at once using a mask
    gray = transforms.functional.rgb_to_grayscale(images, num_output_channels=3)
    gray_mask = (torch.rand(images.shape[0], 1, 1,1, device=device) < 0.1)
    images = torch.where(gray_mask, gray, images)

    # Algorithm 3, Line 19: Images are already resized to 256x256 by the dataloader
    # but we need a 512x512 version for the oracle
    images_oracle = F.interpolate(images, size=(512, 512), mode='bilinear', align_corners=False)

    # Algorithm 3, Lines 21-22: Get normalized oracle features
    with torch.no_grad():
      y_oracle = oracle_extractor(images_oracle)
      y_oracle_normalized = (y_oracle - mu) / sigma

    # Algorithm 3, Line 23: Get teacher features on 256x256 image
    y_teacher = teacher(images)

    # Algorithm 3, Lines 24-28: Compute squared difference and mean loss
    D_dist = (y_oracle_normalized - y_teacher) ** 2
    L_batch = D_dist.mean()
    # End of Pseudocode loop over batch

    # Algorithm 3, Line 29: Update teacher parameters
    optimizer.zero_grad()
    L_batch.backward()
    optimizer.step()
    if iteration % 500 == 0:
      print(f"Iteration {iteration}/{iterations} | Loss: {L_batch.item():.6f}")
```

Listing 2: Student and Autoencoder Training

```python
# Follow Algorithm 1 in the paper to train the autoencoder and student
# As in the paper, need pretrained teacher (we have), sequence of training/validation images (given)

def train_student_and_autoencoder(training_data, validation_data, testing_data, testing_labels):

  # 1. Compute Mu and Sigma based on training data set (Algorithm 1, Steps 3-9)
  teacher_outputs = []
  with torch.no_grad():
    for i in range(len(training_data)):
      # Prepare image and get teacher features
      img = training_data[i:i+1]
      y_prime = teacher(img) # Shape: [1, 384, 64, 64]
      teacher_outputs.append(y_prime)

  # Concatenate all outputs into one large tensor [N, 384, 64, 64]
  all_outputs = torch.cat(teacher_outputs, dim=0)

  # Compute mu and sigma per channel across the entire dataset
  # We reduce over Dim 0 (Images), Dim 2 (Height), and Dim 3 (Width)
  mu = torch.mean(all_outputs, dim=(0, 2, 3), keepdim=True)
  sigma = torch.std(all_outputs, dim=(0, 2, 3), keepdim=True)

```

```python
# Now we initialize a joint Adam optimizer for parameters of student and autoencoder (learning rate and weight decay from algorithm 1)
lr = 1e-4
weight_decay = 1e-5
sa_parameters = list(student.parameters()) + list(autoencoder.parameters())
sa_optimizer = optim.Adam(sa_parameters, lr=lr, weight_decay=weight_decay)

# Initialize pretraining iterator once, outside the main loop.
# We don't want to create a new pretraining iterator instance with each loop iteration
distill_dataset = datasets.ImageFolder(root=os.path.join(path, 'train'), transform=transform)
distill_loader = DataLoader(distill_dataset, batch_size=1, shuffle=True) # add this because it is loading whole thing in teacher now
pretrain_iter = iter(distill_loader)

batch_size = min(8, len(training_data))

# Main training loop
for i in range(1, 70001):
  # Choose 'random' training image, do fwd pass with teacher, do fwd pass with student.
  indices = torch.randint(0, len(training_data), (batch_size,), device=training_data.device)
  random_image = training_data[indices]

  with torch.no_grad():
    teacher_output = teacher(random_image)
  student_output = student(random_image)

  # Get the normalized teacher output per channel
  normalized_teacher_output = (teacher_output - mu) / sigma

  # Now compute square difference between normalized teacher output and the first 384 entries of the student output
  student_output_f384 = student_output[:, :384, :, :]
  diff_st = (normalized_teacher_output - student_output_f384)**2

  # Get the 0.999 quantile of the difference, and compute loss as mean of diff_st entries over or equal to the quantile
  diff_st_999 = torch.quantile(torch.flatten(diff_st), q=0.999)

  # PyTorch boolean indexing rather than slow python loop
  loss_hard = diff_st[diff_st >= diff_st_999].mean()

  # Now choose a random pretraining ImageNet image. Load pretraining images more efficiently by
  # creating an infinite loop of the image data (so that we never run out) with a try block
  # This way we don't need to create a new pretraining iterator with each loop iteration
  try:
    pretraining_image, _ = next(pretrain_iter)
  except StopIteration:
    pretrain_iter = iter(distill_loader)
    pretraining_image, _ = next(pretrain_iter)
  pretraining_image = pretraining_image.to(device)

  # Now get the student teacher loss with hard loss and a regularization term defined in the algorithm
  student_output_pretaining_384 = student(pretraining_image)[:, :384, :, :]
  reg_term = (student_output_pretaining_384**2).mean()
  loss_st = loss_hard + reg_term

  # Now we augment the training images for use on the encoder branch. We do this as we want the encoder to
  # learn more general structures. (Use adjustments from Algorithm 1)
  augmentation_index = np.random.randint(1, 4)
  lam = np.random.uniform(0.8, 1.2)
  if augmentation_index == 1:
    augmented_images = torchvision.transforms.functional.adjust_brightness(random_image, lam)
  elif augmentation_index == 2:
    augmented_images = torchvision.transforms.functional.adjust_contrast(random_image, lam)
  elif augmentation_index == 3:
    augmented_images = torchvision.transforms.functional.adjust_saturation(random_image, lam)

  # With the augmented training images, run forward passes on the autoencoder and the teacher,
  # and normalize as we did before.
  autoencoder_output = autoencoder(augmented_images)
  with torch.no_grad():
    teacher_output_augmented = teacher(augmented_images)

  normalized_teacher_output_augmented = (teacher_output_augmented - mu) / sigma
  student_output_augmented = student(augmented_images)
  student_output_augmented_l384 = student_output_augmented[:, 384:, :, :]
```

```
    diff_ta = (normalized_teacher_output_augmented - autoencoder_output)**2
    diff_sa = (autoencoder_output - student_output_augmented_l384)**2

    # For the loss, we take the sum of the means of each of the differences (teacher-autoencoder, student-autoencoder,
    # and the student-teacher loss from earlier)
    loss_ta = torch.mean(diff_ta)
    loss_sa = torch.mean(diff_sa)
    loss = loss_st + loss_ta + loss_sa

    if i % 500 == 0:
      print("At iteration:", i, " loss: ",  loss)

    # Update the Adam optimizer parameters. Clear old gradients (avoid accumulation), and step
    sa_optimizer.zero_grad()
    loss.backward()
    sa_optimizer.step()

    # Decay learning rate after 4500 iterations
    if i == 66500:
      sa_optimizer.param_groups[0]['lr'] = 1e-5


  # Here, the training of student and autoencoder weights is done. We now make anomaly maps
  # and get anomaly scores
  st_anomaly_scores = []
  sa_anomaly_scores = []

  student.eval()
  autoencoder.eval()
  # Use more normal images from the validation set to get high end differences in normal images
  for i in range(len(validation_data)):
    val_image = validation_data[i:i+1]

    # Get outputs and normalize/compute loss similar to before
    with torch.no_grad():
      teacher_output = teacher(val_image)
    student_output = student(val_image)
    autoencoder_output = autoencoder(val_image)

    student_output_f384 = student_output[:, :384, :, :]
    student_output_l384 = student_output[:, 384:, :, :]
    normalized_teacher_output = (teacher_output - mu) / sigma
    diff_st = (normalized_teacher_output - student_output_f384)**2
    diff_sa = (autoencoder_output - student_output_l384)**2

    # Make anomaly maps using the differences. Take mean difference across channel for each output pixel
    map_st = diff_st.mean(dim=1)
    map_sa = diff_sa.mean(dim=1)

    # Resize with bilinear interpolation (like done in the autoencoder forward pass)
    map_st = F.interpolate(input=map_st.unsqueeze(1), size=(256, 256), mode='bilinear')
    map_sa = F.interpolate(input=map_sa.unsqueeze(1), size=(256, 256), mode='bilinear')

    # Append vectorized anomaly scores to score lists
    st_anomaly_scores.append(torch.flatten(map_st))
    sa_anomaly_scores.append(torch.flatten(map_sa))

  # We now have a list of flattened local (st) anomaly maps, and global (sa)
  # Get quantile values for 0.9 and 0.995 for each map so we have a reference later for what to consider an anomaly
  st_quantile_90 = torch.quantile(torch.cat(st_anomaly_scores), 0.90)
  st_quantile_995 = torch.quantile(torch.cat(st_anomaly_scores), 0.995)

  sa_quantile_90 = torch.quantile(torch.cat(sa_anomaly_scores), 0.90)
  sa_quantile_995 = torch.quantile(torch.cat(sa_anomaly_scores), 0.995)

  return teacher, student, autoencoder, mu, sigma, st_quantile_90, st_quantile_995, sa_quantile_90, sa_quantile_995
```

Listing 3: Inference

```
# Algorithm 2 implementation:
def inference(teacher, student, autoencoder, mu, sigma, st_quantile_95, st_quantile_995, sa_quantile_95, sa_quantile_995, test_image):
  # Get test image and get outputs of each network
```

```
  teacher_output = teacher(test_image)
  student_output = student(test_image)
  autoencoder_output = autoencoder(test_image)

  # Normalize the teacher output and get the differences as in training function
  normalized_teacher_output = (teacher_output - mu) / sigma
  student_output_f384 = student_output[:, :384, :, :]
  student_output_l384 = student_output[:, 384:, :, :]
  diff_st = (normalized_teacher_output - student_output_f384)**2
  diff_sa = (autoencoder_output - student_output_l384)**2

  # Generate anomaly maps as done in training
  map_st = diff_st.mean(dim=1)
  map_sa = diff_sa.mean(dim=1)
  map_st = F.interpolate(input=map_st.unsqueeze(1), size=(256, 256), mode='bilinear')
  map_sa = F.interpolate(input=map_sa.unsqueeze(1), size=(256, 256), mode='bilinear')

  # Normalize the original maps and combine them
  normalized_map_st = 0.1 * (map_st - st_quantile_95) / (st_quantile_995 - st_quantile_95)
  normalized_map_sa = 0.1 * (map_sa - sa_quantile_95) / (sa_quantile_995 - sa_quantile_95)

  combined_map = 0.5 * normalized_map_st + 0.5 * normalized_map_sa
  image_level_score = max(torch.flatten(combined_map))

  return combined_map, image_level_score
```

Listing 4: PDN Class

```
class PDN(nn.Module):
  def __init__(self, out_channels=384, padding=True):
    super(PDN, self).__init__()

    p = 3 if padding else 0

    self.conv1 = nn.Conv2d(3, 128, kernel_size=4, stride=1, padding=p);
    self.avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2, padding=1);
    self.conv2 = nn.Conv2d(128, 256, kernel_size=4, stride=1, padding=p);
    self.avgpool2 = nn.AvgPool2d(kernel_size=2, stride=2, padding=1);
    self.conv3 = nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1);
    self.conv4 = nn.Conv2d(256, out_channels, kernel_size=4, stride=1, padding=0);

    self.relu = nn.ReLU(inplace=True);

  # Compute the forward pass
  def forward(self, x):
    x = self.relu(self.conv1(x))
    x = self.avgpool1(x)
    x = self.relu(self.conv2(x))
    x = self.avgpool2(x)
    x = self.relu(self.conv3(x))
    x = self.conv4(x)
    return x
```

Listing 5: Autoencoder Class

```
class AutoEncoder(nn.Module):
  def __init__(self,):
    super().__init__()

    # Encoder Layers as class attributes. Conv2d docs:
    # https://docs.PyTorch.org/docs/stable/generated/torch.nn.Conv2d.html
    # Input channels is what the layer receives, and output channels is
    # specified in table 8 of the paper.
    # Kernel size, stride, and padding all taken directly from table 8
    self.EncConv1 = nn.Conv2d(in_channels=3, out_channels=32,
                              kernel_size=(4, 4), stride=(2, 2), padding=1)
    self.EncConv2 = nn.Conv2d(in_channels=32, out_channels=32,
                              kernel_size=(4, 4), stride=(2, 2), padding=1)
    self.EncConv3 = nn.Conv2d(in_channels=32, out_channels=64,
                              kernel_size=(4, 4), stride=(2, 2), padding=1)
    self.EncConv4 = nn.Conv2d(in_channels=64, out_channels=64,
                              kernel_size=(4, 4), stride=(2, 2), padding=1)
    self.EncConv5 = nn.Conv2d(in_channels=64, out_channels=64,
                              kernel_size=(4, 4), stride=(2, 2), padding=1)
    self.EncConv6 = nn.Conv2d(in_channels=64, out_channels=64,
```

```
                                kernel_size=(8, 8), stride=(1, 1), padding=0)

    # Decoder layers (more Conv2D)
    self.DecConv1 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv2 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv3 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv4 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv5 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv6 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(4, 4), stride=(1, 1), padding=2)
    self.DecConv7 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=(3, 3), stride=(1, 1), padding=1)
    self.DecConv8 = nn.Conv2d(in_channels=64, out_channels=384,
                                kernel_size=(3, 3), stride=(1, 1), padding=1)

    # Bilinear Layers: https://docs.PyTorch.org/docs/stable/generated/torch.nn.Upsample.html
    self.Bilinear1 = nn.Upsample(size=(3, 3), mode="bilinear")
    self.Bilinear2 = nn.Upsample(size=(8, 8), mode="bilinear")
    self.Bilinear3 = nn.Upsample(size=(15, 15), mode="bilinear")
    self.Bilinear4 = nn.Upsample(size=(32, 32), mode="bilinear")
    self.Bilinear5 = nn.Upsample(size=(63, 63), mode="bilinear")
    self.Bilinear6 = nn.Upsample(size=(127, 127), mode="bilinear")
    self.Bilinear7 = nn.Upsample(size=(64, 64), mode="bilinear")

    # Dropout Layers: https://docs.PyTorch.org/docs/stable/generated/torch.nn.Dropout.html
    self.Dropout1To6 = nn.Dropout(p=0.2) # All dropout layers have the same p value

    # Relu activation function (for convenience)
    self.relu = nn.ReLU()

  def forward(self, x):
    # Compute a forward pass. Here we just combine the layers in the order of table 8 in the EfficientAD
    ↪ paper
    out = self.relu(self.EncConv1(x))
    out = self.relu(self.EncConv2(out))
    out = self.relu(self.EncConv3(out))
    out = self.relu(self.EncConv4(out))
    out = self.relu(self.EncConv5(out))
    out = self.EncConv6(out)

    out = self.Bilinear1(out)

    out = self.relu(self.DecConv1(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear2(out)

    out = self.relu(self.DecConv2(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear3(out)

    out = self.relu(self.DecConv3(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear4(out)

    out = self.relu(self.DecConv4(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear5(out)

    out = self.relu(self.DecConv5(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear6(out)

    out = self.relu(self.DecConv6(out))
    out = self.Dropout1To6(out)
    out = self.Bilinear7(out)

    out = self.relu(self.DecConv7(out))
    out = self.DecConv8(out)

    return out
```

REFERENCES

[1] K. Batzner, L. Heckler, and R. König, "EfficientAD: Accurate visual anomaly detection at millisecond-level latencies," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024, pp. 128–138.

[2] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," *arXiv preprint arXiv:2106.08265*, 2021.

[3] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings," *arXiv preprint arXiv:1911.02357*, 2020.

[4] Yanivbohbot, "Anomaly detection with isolation forest: A complete guide," Medium, Jul. 2025, [Online]. Available: https://medium.com/@yanivbohbot5/anomaly-detection-with-isolation-forest-a-complete-guide-d42da77510a6.

[5] D. Cochard, "PaDiM: A machine learning model for detecting defective products without retraining," Medium, Oct. 2021, [Online]. Available: https://medium.com/axinc-ai/padim-a-machine-learning-model-for-detecting-defective-products-without-retraining-5daa6f203377.